```cpp
// ─────────────────────────────────────────────────────────────────────
//  COMPLETE SOLUTION: Dynamic Programming + Frame-Stewart Move Generator
//  Computes the minimum moves table, prints the DP table, answers the
//  '33 moves' question, then generates and prints the full move sequence.
// ─────────────────────────────────────────────────────────────────────
#include <iostream>
#include <vector>
#include <string>
#include <climits>
#include <iomanip>  // for setw()
using namespace std;
typedef long long ll;
```

```cpp

// ── Global move log ──────────────────────────────────────────────
int        moveCount = 0;
vector<string> moveLog;

void logMove(int disk, char from, char to) {
    ++moveCount;
    string moveStr;
    if (moveCount < 10)
        moveStr = "  Move " + to_string(moveCount) + ":  ";
    else
        moveStr = "  Move " + to_string(moveCount) + ": ";

    string diskStr = "Disk " + to_string(disk);
    if (disk < 10) diskStr += " ";

    moveLog.push_back(moveStr + diskStr + "  " + from + " -> " + to);
}
```

```
31
32  // ── Three-peg Hanoi (used as Step-2 subroutine) ──────────────────────
33  void hanoi3(int n, int base, char from, char to, char aux) {
34      if (n == 0) return;
35      hanoi3(n - 1, base, from, aux, to);
36      logMove(base + n - 1, from, to);
37      hanoi3(n - 1, base, aux, to, from);
38  }
```

```cpp
// ── Frame-Stewart four-peg D&C ──────────────────────────────────
void hanoi4(int n, int base, char from, char to,
            char aux1, char aux2,
            const vector<int>& kopt) {
    if (n == 0) return;
    if (n == 1) { logMove(base, from, to); return; }

    int k = kopt[n];
    hanoi4(k, base, from, aux1, aux2, to, kopt);  // Step 1
    hanoi3(n - k, base + k, from, to, aux2);              // Step 2
    hanoi4(k, base, aux1, to, from, aux2, kopt); // Step 3
}
```

```cpp
// ── DP: build Frame-Stewart table for 1..N ──────────────────────────
void buildDP(int N, vector<ll>& dp, vector<int>& kopt) {
    dp.assign(N + 1, LLONG_MAX);
    kopt.assign(N + 1, 0);
    dp[0] = 0; dp[1] = 1; kopt[1] = 1;

    for (int i = 2; i <= N; i++) {
        for (int k = 1; k < i; k++) {
            ll h3 = (1LL << (i - k)) - 1;   // T(i-k, 3)
            ll cost = 2LL * dp[k] + h3;
            if (cost < dp[i]) {
                dp[i] = cost;
                kopt[i] = k;
            }
        }
    }
}
```

```
70  // ── Print a separator line ──────────────────────────────────────────
71  void printSeparator(int nWidth, int t4Width, int kWidth, int t3Width) {
72      cout << "+";
73      for (int i = 0; i < nWidth + 2; i++) cout << "-";
74      cout << "+";
75      for (int i = 0; i < t4Width + 2; i++) cout << "-";
76      cout << "+";
77      for (int i = 0; i < kWidth + 2; i++) cout << "-";
78      cout << "+";
79      for (int i = 0; i < t3Width + 2; i++) cout << "-";
80      cout << "+" << endl;
81  }
```

```cpp
// ── Main ──────────────────────────────────────────────────────────
int main() {
    const int N = 8;
    vector<ll>  dp;
    vector<int> kopt;
    buildDP(N, dp, kopt);

    // ── Print DP table ────────────────────────────────────────────
      // ── Calculate column widths ──
    int nWidth = to_string(N).length();
    int t4Width = to_string(dp[N]).length();
    int kWidth = to_string(kopt[N]).length();
    int t3Width = to_string((1LL << N) - 1).length();

    // ── Minimum widths for headers ──
    if (nWidth < 1) nWidth = 1;
    if (t4Width < 6) t4Width = 6;   // "T(n,4)"
    if (kWidth < 4) kWidth = 4;     // "kopt"
    if (t3Width < 6) t3Width = 6;   // "T(n,3)"

    // ── Print DP table with proper formatting ──
    printSeparator(nWidth, t4Width, kWidth, t3Width);
    cout << "| " << left << setw(nWidth) << "n"
        << " | " << setw(t4Width) << "T(n,4)"
        << " | " << setw(kWidth) << "kopt"
        << " | " << setw(t3Width) << "T(n,3)" << " |" << endl;
    printSeparator(nWidth, t4Width, kWidth, t3Width);

    for (int i = 1; i <= N; i++) {
        cout << "| " << left << setw(nWidth) << i
            << " | " << setw(t4Width) << dp[i]
            << " | " << setw(kWidth) << kopt[i]
            << " | " << setw(t3Width) << ((1LL << i) - 1) << " |" << endl;
    }
    printSeparator(nWidth, t4Width, kWidth, t3Width);

    // ── Answer the 33-move question ──────────────────────────────
    cout << "\nOPT(8,4) = " << dp[N] << endl;
    cout << "Can DP solve in 33 moves? "
        << (dp[N] == 33 ? "YES - confirmed!" : "NO") << endl;

    // ── Generate the full move sequence ──────────────────────────
    hanoi4(N, 1, 'A', 'D', 'B', 'C', kopt);
    cout << "\nTotal moves generated: " << moveCount << endl;
    for (auto& m : moveLog) cout << m << endl;

    return 0;
}
```

n	T(n,4)	kopt	T(n,3)
1	1	1	1
2	3	1	3
3	5	1	7
4	9	1	15
5	13	2	31
6	17	3	63
7	25	3	127
8	33	4	255

```
OPT(8,4) = 33
Can DP solve in 33 moves? YES - confirmed!

Total moves generated: 33
  Move 1:  Disk 1   A -> C
  Move 2:  Disk 2   A -> B
  Move 3:  Disk 3   A -> D
  Move 4:  Disk 2   B -> D
  Move 5:  Disk 4   A -> B
  Move 6:  Disk 2   D -> A
  Move 7:  Disk 3   D -> B
  Move 8:  Disk 2   A -> B
  Move 9:  Disk 1   C -> B
  Move 10: Disk 5   A -> C
  Move 11: Disk 6   A -> D
  Move 12: Disk 5   C -> D
  Move 13: Disk 7   A -> C
  Move 14: Disk 5   D -> A
  Move 15: Disk 6   D -> C
  Move 16: Disk 5   A -> C
  Move 17: Disk 8   A -> D
  Move 18: Disk 5   C -> D
  Move 19: Disk 6   C -> A
  Move 20: Disk 5   D -> A
  Move 21: Disk 7   C -> D
  Move 22: Disk 5   A -> C
  Move 23: Disk 6   A -> D
  Move 24: Disk 5   C -> D
  Move 25: Disk 1   B -> A
  Move 26: Disk 2   B -> D
  Move 27: Disk 3   B -> C
  Move 28: Disk 2   D -> C
  Move 29: Disk 4   B -> D
  Move 30: Disk 2   C -> B
  Move 31: Disk 3   C -> D
  Move 32: Disk 2   B -> D
  Move 33: Disk 1   A -> D
```